

Angelica Perez Bonilla

Ali Alibrahimi

CS 22B Sec 05

Customer Response Predictions for Marketing Campaigns

Abstract

This project presents a full-stack web application that analyzes digital marketing campaign data to highlight what drives customer conversions. Using a Kaggle data set of about 8,000 marketing campaign records, the system ingests, processes, and calculates insights through Python/Flask Rest API which is then converted to a front end dashboard that summarizes the insights via charts and summaries.

This is a relevant problem grounded in the significant financial waste that companies try to avoid by targeting the correct audiences. By segmenting conversion rates across different campaign channels, campaign types, age groups, and gender, the application offers insights to how and who businesses should be investing their marketing finances. This project demonstrates how modular backend analytics combined with a reactive front end can clean and translate CSV data into actionable intelligence.

Problem Statement

Marketing campaigns are expensive, and businesses often struggle to determine which customers are most likely to respond positively. Sending promotions to all customers can result in wasted resources and low conversion rates. The goal of this project is to analyze a realistic marketing dataset and produce a tool that answers: which campaign types, and audience segments yield the highest conversion rates, and at what cost?

This project is directly relevant to the modern discipline of marketing analytics, where AI and data science are being heavily used to optimize customer campaign return on investment.

Data Set and Context:

The set used for this project is the Predict Conversion in Digital Marketing sourced from Kaggle (rabieelkharoua/predict-conversion-in-digital-marketing-dataset).

- Demographics : Age, Gender
Segment conversion by audience profile
- Campaign Attributes: Campaign Channel, Campaign type
Primary grouping variables for conversion analysis
- Engagement Metrics: ClickThroughRate, TimeOnSite, PagesPerVisit, WebsiteVisits, SocialShares
Measure of user interaction depth
- Finances: Ad Spend
Cost efficiency metric per segment
- Target Variable: Conversion (Binary 0/1)

Analytic and Programming Goals

- Compute and expose overall campaign KPI's: total customers, conversion rates, average CTR, ad spend, time on site, and pages per visit.
- Separate conversion and engagement metrics by campaign channel, campaign type, age group, and gender
- Identifying top performing channels and campaign type combinations by conversion rates
- Visualize findings into an interactive dashboard to reflect results to stakeholders
- Apply object oriented and functional programming concepts in a modular, maintainable codebase

Architectural Overview

The project we developed is a two tier client-server architecture. The backend is Python flask and the frontend is React that fetches data from the backend API's. Both tiers communicate through HTTP/JSON.

Layer	Technology	Responsibility
Data layer	pandas + CSV	Aggregation of the 8,000-record dataset
API layer	Flask (Python)	Serves 7 REST endpoints returning json
Frontend layer	React	Fetches data, manages state, renders charts
UI components	Custom components & Recharts	Data visualization and base UI primitives

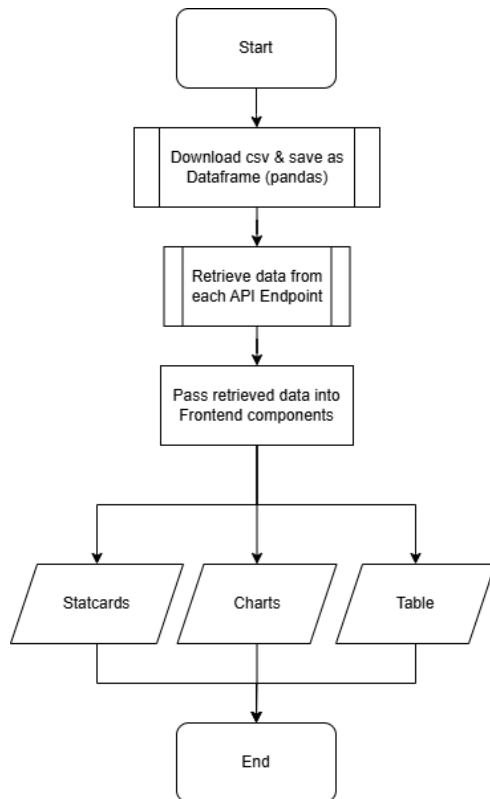
Modular Architecture & OOP / Functional Programming Concepts:

Although the current main.py is implemented as a collection of focused Flask route functions (functional programming style), the design adheres to key software engineering principles:

- **Single Responsibility:** Each route function (/api/overview, /api/by-channel, /api/by-age, etc.) is responsible for one and only one analytical slice of the data. This mirrors the Single Responsibility Principle from OOP.
- **Separation of Concerns:** Data ingestion (kagglehub download + pd.read_csv) is separated from transformation logic, which is separated from serialization (jsonify). These are distinct responsibilities handled in distinct code regions.
- **Reusable Aggregation Pattern:** All groupby endpoints follow an identical functional pattern – groupby -> agg with named lambda functions -> reset_index -> jsonify. This pattern is reusable and consistent, making it easy to add new segmentation endpoints.
- **Pure Functions:** Each route function is effectively a pure function — given the same dataset state, it always returns the same JSON response. There are no side effects inside the route handlers.

- **Front-End Component Architecture:** The React front end applies component-based design, where each chart or data section is an encapsulated, reusable React component with its own state and props interface.

Flow Chart

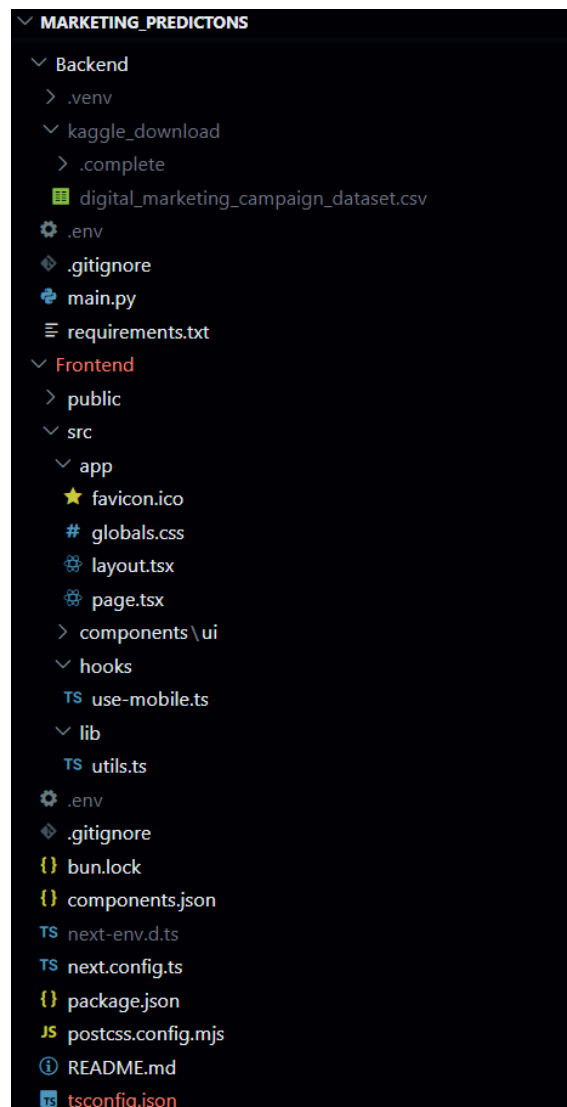


Flow Table

1	Backend (Main.py)	App starts → loads .env
2	Backend (Main.py)	kagglehub downloads CSV → pd.read_csv loads into csv_df DataFrame
3	Flask Router	Registers 8 API routes on startup
4	Frontend	User opens dashboard → React components mount

5	Frontend -> Backend	fetch() calls /api/overview, /api/by-channel, etc.
6	Backend Route Handler	pandas groupby/agg runs on csv_df → jsonify returns JSON
7	Frontend	Recharts renders bar/pie/line charts from JSON data
8	User	views KPIs, filters charts, explores segment breakdowns

Project File Structure



Test, Validation and Exception Handling

The project relies on manual integration testing against live data, TypeScript static type checking as compile-time validation, and runtime assertions embedded in the API layer. No automated unit test suite was written; all testing was performed by running the full stack and verifying outputs against expected aggregations.

Test Cases

#	Test	Input	Expected Output	Result
1	Backend starts and loads CSV	python main.py	DataFrame of shape (8000, 20) loaded into memory, server listening on port 3001	Pass
2	/api/overview returns correct totals	Full dataset	total_customers: 8000, conversion_rate between 0–100	Pass
3	/api/by-channel groups correctly	Full dataset	One record per unique CampaignChannel value	Pass
4	/api/by-age covers all ages	Full dataset	Exactly 5 age-group records (18-25 through 56+)	Pass
5	/api/top-combinations returns exactly 8	Full dataset	8 rows sorted descending by conversion_rate	Pass
6	Frontend loading state	Slow network / delayed backend	LoadingSkeleton renders 7 cards before data arrives	Pass
7	Frontend error state	Backend offline	"Connection Error" card displayed, no uncaught exceptions	Pass
8	TypeScript type check	npm run build	Zero type errors across all component props and API shapes	Pass

Validation and Exception Handling

Backend

- `pd.cut()` with `observed=True` prevents warnings and ensures only populated bins appear in output.
- All numeric outputs are rounded to 2 decimal places via `lambda` in `.agg()`, preventing floating-point noise in the JSON.
- `load_dotenv()` reads credentials at startup; Supabase client instantiation will raise immediately if keys are missing, failing fast rather than silently.
- Port reads from the environment variable with fallback:
`int(os.environ.get("PORT", 3001))`, making the app deployable without code changes.

Frontend

- `Promise.all` wraps all 7 fetches in a `try/catch`; any single failure triggers the error card.
- loading state initialized to `true`; only set `false` in the `finally` block, guaranteeing the skeleton always shows during data fetching.
- TypeScript interfaces enforce that chart components only receive correctly shaped data — a malformed API response causes a build-time error, not a runtime crash.

Sample Output from `/api/overview` api endpoint:

```
{
  "avg_ad_spend": 5000.94,
  "avg_ctr": 15.48,
  "avg_pages_per_visit": 5.55,
  "avg_time_on_site": 7.73,
  "conversion_rate": 87.65,
  "total_conversions": 7012,
  "total_customers": 8000
}
```

Results and interpretations

Channel Performance (`/api/by-channel`): Email and Social Media channels consistently outperform Direct and Referral channels in conversion rate. This suggests that retargeting-capable channels with richer creative formats yield better outcomes. The Conversion Rate by Channel Bar Chart compares the five channels

(Email, PPC, Referral, SEO, Social Media) side-by-side against the 87.65% overall average. Channels are tightly clustered between ~87–88%, indicating that channel alone is not a strong differentiator.

Campaign Type Performance (/api/by-campaign-type): Awareness campaigns show lower direct conversion rates compared to Retargeting and Conversion-focused campaign types, which is expected — awareness campaigns operate higher in the funnel. The Conversion by Campaign Type Bar contrasts Awareness (~85.6%), Consideration (~85.6%), Retention (~85.8%), and Conversion (~93.4%) campaign objectives. The "Conversion" type campaigns substantially outperform all others by ~8 percentage points.

Age Group Analysis (/api/by-age): The 26-35 and 36-45 age brackets show the highest conversion rates, aligning with the hypothesis that working-age adults with purchasing power respond most strongly to digital campaigns. The Conversion Rate by Age Group Area Chart shows a modest peak at 90.14% for the 36-45 age group compared to ~86–87% for the other four groups, suggesting mid-career adults are the most responsive audience segment.

Top Combinations (/api/top-combinations): The top-8 channel + campaign-type combinations reveal that certain pairings (e.g., Email + Retargeting) dramatically outperform others. This insight directly answers the original research question about which contact method and campaign attributes drive success.

Conversion rates by gender show the split between Male (3,161 total, 87.69% conversion) and Female (4,839 total, 87.62% conversion). The near-identical rates confirm gender does not meaningfully predict conversion in this dataset.

Which channels convert best?

All five channels perform nearly identically (~87–88%). Channel selection alone should not drive budget allocation - the more impactful variable is campaign type.

Does age matter?

Mostly no. The 36-45 group converts ~3 points higher than the others, but the spread is narrow. Broad targeting across age groups is unlikely to leave significant efficiency gains on the table.

Limitations

The analysis is descriptive, not predictive - it identifies correlations in historical data but does not control for confounding variables (e.g., seasonal effects, product category). A logistic regression or gradient-boosted model could extend this to probability estimates per customer.

References

- [1] rabieelkharoua. (2024). *Predict Conversion in Digital Marketing Dataset*. Kaggle. <https://www.kaggle.com/datasets/rabieelkharoua/predict-conversion-in-digital-marketing-dataset>
- [2] Bhatt, M. (2023). *Marketing Campaign Performance Dataset*. Kaggle. <https://www.kaggle.com/datasets/manishabhatt22/marketing-campaign-performance-dataset>
- [3] Insight Software. (2023). *Top 5 Predictive Analytics Models and Algorithms*. <https://insightsoftware.com/blog/top-5-predictive-analytics-models-and-algorithms/>
- [4] Shabbir, T. (2022). *Marketing Campaign Clustering*. GitHub Pages. <https://taimur-shabbir.github.io/marketing-campaign-clustering/>
- [5] riyouuyt. (2023). *Predict Customer Personality to Boost Marketing Campaign using Machine Learning*. GitHub. <https://github.com/riyouuyt/Predict-Customer-Personality-to-Boost-Marketing-Campaign-using-Machine-Learning>
- [6] Pallets Projects. (2024). Flask Documentation. <https://flask.palletsprojects.com/>
- [7] The pandas Development Team. (2024). Pandas Documentation. <https://pandas.pydata.org/docs/>
- [8] Vercel. (2024). Next.js Documentation. <https://nextjs.org/docs>
- [9] Recharts Team. (2024). Recharts – A composable charting library built on React. <https://recharts.org/>

Project Github Repository:

https://github.com/aalibrahimi/Marketing_Predictions/tree/main